



Application Development

Android Studio · Kotlin · Jetpack Compose

A teachable studio curriculum for building real Android applications

Teacher guidance, student explanations, annotated code, practical labs, assessments, and release practice.

Programme

Software Engineering

Course

Application Development

Level

University foundation / applied diploma

Format

Lectures · live coding · studio labs · capstone

Suggested duration

12–14 weeks · 2 lectures + 1 lab weekly

Primary stack

Kotlin, Android Studio, Jetpack Compose, Room, Navigation

What this pack is for

This is not a collection of disconnected tutorials. It is a route from environment setup to a tested, accessible, data-backed Android application. Each unit gives the teacher a facilitation plan and gives the student a concrete artefact to produce. The project grows one reliable layer at a time.

Inside the course pack

1	Course orientation	1
1.1	Learning outcomes	1
1.2	Suggested course rhythm	1
1.3	The capstone application	2
2	Unit 1: Android Studio and the Android project	2
2.1	Unit purpose	2
2.2	Teacher preparation	2
2.3	Student explanation: project anatomy	2
2.4	The first project	2
2.5	Lab 1 · Setup evidence	3
2.6	Troubleshooting clinic	3
2.7	Unit 1 check	3
3	Unit 2: Kotlin foundations for Android	4
3.1	Why Kotlin matters	4
3.2	Variables, expressions, and functions	4
3.3	Null safety	4
3.4	Control flow and collections	4
3.5	Data classes and sealed state	5
3.6	Lab 2 · Kotlin practice set	5
3.7	Unit 2 check	5
4	Unit 3: Compose fundamentals and Android Studio UI tools	6
4.1	The Compose mental model	6
4.2	Composable functions and previews	6
4.3	Modifiers as readable decisions	6
4.4	Layout and Material 3	7
4.5	Unit 3 check	7
5	Unit 4: State, events, forms, and ViewModel	8
5.1	State is the present truth	8
5.2	State hoisting	8
5.3	ViewModel and unidirectional data flow	8
5.4	Lab 4 · Form state machine	9
5.5	Unit 4 check	9
6	Unit 5: Navigation and application architecture	10
6.1	Navigation is a product decision	10
6.2	Layers and responsibility	11
6.3	Unit 5 check	11
7	Unit 6: Coroutines, Flow, and network data	12
7.1	Asynchronous work is normal	12
7.2	API contract and serialization	12
7.3	The five-state screen	13
7.4	Lab 6 · Remote catalogue	13
7.5	Unit 6 check	13
8	Unit 7: Room persistence and offline-first thinking	14
8.1	Why local storage matters	14

8.2 Repository pattern	14
8.3 Lab 7 · Offline lesson tracker	15
8.4 Unit 7 check	15
9 Unit 8: Material 3, adaptive layouts, and design systems	16
9.1 Visual systems reduce accidental decisions	16
9.2 Adaptive thinking	16
9.3 Lab 8 · Adaptive redesign	17
9.4 Unit 8 check	17
10 Unit 9: Dependency injection and reusable data boundaries	18
10.1 Why injection matters	18
10.2 Hilt-shaped composition root	18
10.3 Lab 9 · Replaceable collaborators	18
10.4 Unit 9 check	18
11 Unit 10: Background work with WorkManager	19
11.1 Work that outlives a screen	19
11.2 Lab 10 · Reliable refresh	19
11.3 Unit 10 check	19
12 Unit 11: Testing, debugging, and performance	20
12.1 A layered quality strategy	20
12.2 Unit test example	20
12.3 Compose UI test example	20
12.4 Debugging protocol	21
12.5 Performance conversation	21
12.6 Lab 11 · Evidence-led repair	21
12.7 Unit 11 check	21
13 Unit 12: Accessibility, privacy, and secure application behaviour	22
13.1 Accessibility is a design requirement	22
13.2 Privacy and authorisation	22
13.3 Lab 12 · Accessibility and privacy audit	22
13.4 Unit 12 check	23
14 Unit 13: Capstone planning and product discovery	24
14.1 Build one useful layer at a time	24
14.2 One user, one job, one proof	24
14.3 Product requirements and acceptance criteria	24
14.4 Story mapping	24
14.5 Lab 13 · Proposal clinic	24
14.6 Unit 13 check	24
15 Unit 14: Capstone implementation, review, and release	25
15.1 Implementation sequence	25
15.2 Code review protocol	25
15.3 Demonstration structure	25
15.4 Release checklist	25
15.5 Final assessment rubric	25
15.6 Capstone submission package	26
16 Teacher field guide	27
16.1 A repeatable lesson pattern	27
16.2 Questions that teach engineering	27
16.3 Common misconceptions	27

16.4 Feedback language	27
17 Student worksheets and lab checklists	28
17.1 Architecture worksheet	28
17.2 Screen-state worksheet	28
17.3 Pull-request checklist	28
17.4 Studio reflection prompts	28
18 Glossary	28
19 References and further study	29
20 Closing note	29

How to use these notes

Teachers can teach each unit in one lecture and one guided studio, then assign the independent task for completion before the next class. Students should read the concept explanation before lab time, type the examples rather than copy them blindly, record questions in the margin, and commit their working project frequently. The pack assumes that understanding is demonstrated through a running application, a clear explanation, and evidence from tests.

Course contract	Teacher responsibility	Student responsibility
Before class	Prepare the environment, identify the live-coding checkpoint, and anticipate one likely misconception.	Read the unit, install or verify tools, and write one question before the session.
During class	Model the reasoning, pause for predictions, and make errors visible without hiding the repair.	Type, predict, test, explain, and ask for clarification when an assumption breaks.
In the lab	Coach through questions; do not take over the keyboard for the student.	Produce the named artefact and record evidence that it works.
After class	Give feedback on decisions, not only syntax.	Submit code, screenshots or test output, and a short reflection.

1 Course orientation

The LEA Labs promise

Application development is the practice of turning a human need into a reliable digital product. The technology matters, but a good engineer also explains the user problem, chooses an appropriate boundary, makes state visible, handles failure, tests the experience, and knows what must improve next. By the end of this course, every learner should be able to open Android Studio, build with Kotlin and Jetpack Compose, store and retrieve data, test a meaningful journey, and defend the decisions behind the result.

1.1 Learning outcomes

By the end of the course, a learner should be able to:

- configure Android Studio, an SDK, an emulator, and a physical test device;
- write readable Kotlin using functions, null safety, data classes, collections, lambdas, and coroutines;
- explain an Android project's Gradle, manifest, source, resource, and test structure;
- construct interfaces with Jetpack Compose, Material 3, modifiers, layouts, lists, forms, and adaptive patterns;
- model UI state and events using state hoisting, ViewModel, and unidirectional data flow;
- navigate between screens and preserve meaningful state across configuration changes;
- separate UI, domain, and data responsibilities with repositories and data sources;
- persist structured data locally with Room and expose it through Flow;
- connect to a network service with loading, empty, error, retry, and authentication states;
- write unit, Compose UI, accessibility, and scenario-based tests;
- apply accessibility, privacy, security, performance, and release-readiness checks;
- present a working capstone with a README, test evidence, known limitations, and a next-step plan.

1.2 Suggested course rhythm

Week	Unit	Guided focus	Evidence
1	Environment and Kotlin	Android Studio, project anatomy, Kotlin syntax, debugging	Running project + Kotlin exercises
2	Compose foundations	Composable functions, modifiers, layout, Material 3	Static multi-screen UI
3	State and interaction	Events, forms, state hoisting, validation, ViewModel	Interactive form flow
4	Navigation and architecture	Navigation graph, UDF, repositories, screen state	Architecture map + navigation
5	Network data	HTTP, serialization, coroutines, Flow, resilient states	Remote data screen
6	Local persistence	Room entities, DAO, database, migrations	Offline-first feature
7	Dependency injection and background work	Hilt concepts, WorkManager, notifications	Scheduled work prototype
8	Quality engineering	Testing, accessibility, performance, profiling	Test matrix + fixes
9–12	Capstone studio	Plan, build, review, improve, release	Final app + defence

1.3 The capstone application

The examples use a consistent fictional product called **LEA Connect**: a learning companion where a learner can view courses, bookmark a lesson, submit a tutor request, and receive a confirmed class. Teachers may replace this domain with a library, clinic, small-business, transport, community, or education idea. The value is in the architecture and reasoning, not the brand-specific data.

Teacher decision

Decide whether the class will build one shared LEA Connect product or allow teams to choose a domain after Unit 4. A shared product reduces setup overhead; separate products increase product-thinking practice. In either case, require the same evidence standard: clear scope, working core journey, failure states, tests, accessibility notes, and a release explanation.

2 Unit 1: Android Studio and the Android project

2.1 Unit purpose

Students need a stable mental model before they write their first screen. Android Studio is the workspace; Gradle describes how the project is built; the Android SDK supplies platform tools; the emulator or device supplies a runtime. The first goal is not visual polish. It is a reproducible “open, sync, run, inspect” loop.

2.2 Teacher preparation

Before the first class, confirm that each student can install or access Android Studio, has at least one API image available, and can create a project from the Empty Activity template. For low-resource machines, prepare a physical-device route and a shared troubleshooting sheet. Ask students to capture the versions of Android Studio, Kotlin, compile SDK, and emulator image in their README.

2.3 Student explanation: project anatomy

An Android project is a set of conventions that lets the IDE and build system find source code, resources, tests, and generated outputs. The exact template changes over time, but the responsibilities remain stable.

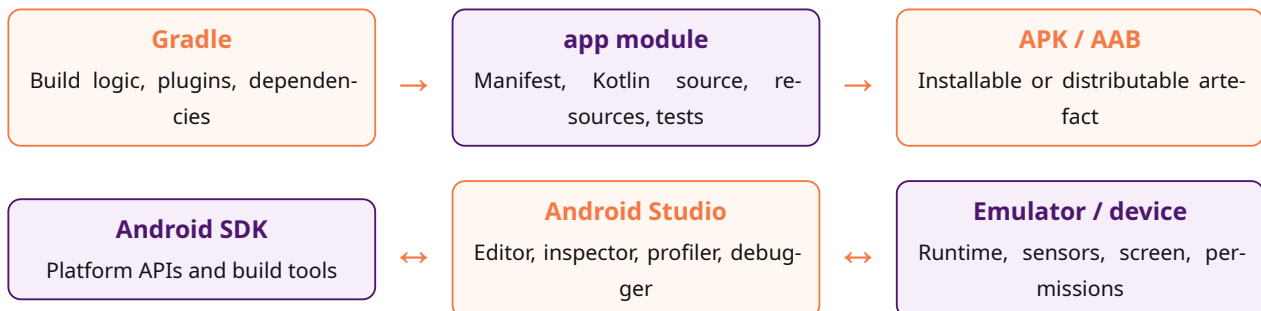


Figure 1: The working parts of an Android Studio project.

2.4 The first project

Use the following sequence in class: create a project, choose Kotlin, choose Jetpack Compose, name the package deliberately, allow Gradle sync to finish, run the app, change one visible string, run again, set one breakpoint, and inspect Logcat. Students should learn that the IDE is an instrument for observing a program, not merely a place to type.

```

package com.example.leaconnect

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
  
```

```
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            WelcomeMessage(name = "LEA learner")
        }
    }
}

@Composable
fun WelcomeMessage(name: String) {
    Text(text = "Welcome, $name")
}
```

Live-coding checkpoint

Stop after the first successful run. Ask students to predict what will happen if `WelcomeMessage` is called with a different name. Then change the value, run again, and explain the path from Kotlin source to rendered text. If students cannot describe that path, do not move on to theming yet.

2.5 Lab 1 · Setup evidence

Students submit a one-page setup record containing: operating system; Android Studio version; compile SDK; emulator or device model; a screenshot of the running app; one Logcat observation; and a paragraph explaining the difference between source code, a build artefact, and a runtime.

2.6 Troubleshooting clinic

| Symptom | Likely cause | Teaching response | |—|—|—| | Gradle sync never finishes | Network, proxy, incompatible plugin, or missing SDK | Read the first error, record versions, and retry with a known-good environment. | | Emulator is slow | Virtualization disabled or machine resources are limited | Use a physical device, reduce AVD resolution, or use a cloud environment. | | App installs but shows an old screen | Wrong run configuration or stale build | Clean only when needed; verify the selected module and change a visible string. | | Unresolved reference | Missing import or dependency | Use IDE suggestions, then explain which module owns the symbol. |

2.7 Unit 1 check

A student passes when they can create a project without a step-by-step rescue, explain the role of Gradle and the SDK, run on an emulator or device, and use a breakpoint or Logcat to inspect behaviour.

3 Unit 2: Kotlin foundations for Android

3.1 Why Kotlin matters

Kotlin is not decoration around Android APIs. It shapes how students model data, express decisions, handle missing values, structure asynchronous work, and communicate intent. Teach syntax through small problems, then connect each concept to an application need.

3.2 Variables, expressions, and functions

```
fun calculateTotal(price: Double, quantity: Int): Double {
    require(price >= 0) { "Price cannot be negative" }
    require(quantity >= 0) { "Quantity cannot be negative" }
    return price * quantity
}

val total = calculateTotal(price = 2500.0, quantity = 2)
```

Explain `val` as a read-only reference, `var` as a changeable reference, named arguments as readable intent, and `require` as a boundary check. Students should practise turning vague rules into named functions with one responsibility.

3.3 Null safety

Mobile applications meet incomplete data constantly: a profile may have no photo, a network response may omit an optional field, and a database record may not exist yet. Kotlin's type system makes absence visible.

```
data class Learner(
    val id: String,
    val name: String,
    val preferredName: String?
)

fun greeting(learner: Learner): String {
    val nameToShow = learner.preferredName ?: learner.name
    return "Hello, $nameToShow"
}
```

The safe-call operator `?.` and Elvis operator `?:` are useful, but they should not hide an important error. Teach students to distinguish an optional value from a value that must exist for the operation to be safe.

3.4 Control flow and collections

```
data class Course(val id: String, val title: String, val priceKes: Int)

val courses = listOf(
    Course("android", "Application Development", 45000),
    Course("web", "Web Development", 45000),
    Course("ai", "Applied AI", 35000),
)
```



```
val affordable = courses.filter { it.priceKes <= 45000 }  
val titles = affordable.map { it.title }
```

Ask students to read the code aloud. `filter` answers “which items?”, `map` answers “what representation?”, and `sortedBy` answers “in what order?”. The point is not memorising functions; it is making the transformation legible.

3.5 Data classes and sealed state

```
sealed interface ScreenState {  
    data object Idle : ScreenState  
    data object Loading : ScreenState  
    data class Success(val courses: List<Course>) : ScreenState  
    data class Error(val message: String) : ScreenState  
}
```

A sealed state model prevents accidental combinations such as “loading and error and success at the same time.” It also makes `when` expressions exhaustive.

Teacher move · Predict before compile

Present a function with one missing type or branch. Ask pairs to predict the compiler message and propose a repair before running the code. This changes the compiler from a source of shame into a feedback partner.

3.6 Lab 2 · Kotlin practice set

Students create a command-line Kotlin file or Android unit-test file containing: a price calculator with validation; a function that safely formats an optional phone number; a data class for a lesson; a filter that finds unfinished lessons; and a sealed result model. Require tests for valid input, invalid input, empty collections, and missing optional values.

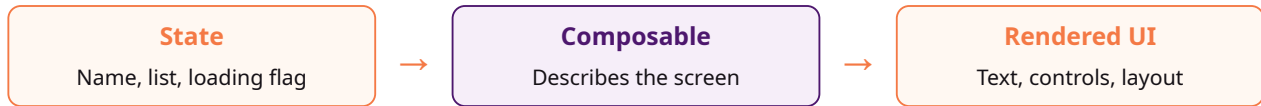
3.7 Unit 2 check

A student passes when they can explain the difference between nullable and non-nullable values, use a data class for domain data, transform a collection without mutating it accidentally, and model success and failure states explicitly.

4 Unit 3: Compose fundamentals and Android Studio UI tools

4.1 The Compose mental model

Jetpack Compose is declarative: the UI is described as a function of current state. Instead of manually finding a view and changing its text, a composable receives data and describes what should be visible. When state changes, Compose recomposes the affected parts.



User event changes state; Compose describes the new UI.

Figure 2: The declarative UI loop.

4.2 Composable functions and previews

```

@Composable
fun CourseCard(
    course: Course,
    onOpen: (String) -> Unit,
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable { onOpen(course.id) }
            .semantics { contentDescription = "Open ${course.title}" }
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(text = course.title, style = MaterialTheme.typography.titleMedium)
            Text(text = "KES ${course.priceKes}")
        }
    }
}

@Preview(showBackground = true)
@Composable
private fun CourseCardPreview() {
    CourseCard(
        course = Course("android", "Application Development", 45000),
        onOpen = {},
    )
}
  
```

Explain the contract: `CourseCard` receives data and emits an event. It does not navigate, fetch data, or decide who may enrol. This makes it reusable and testable.

4.3 Modifiers as readable decisions

A modifier is a chain of layout, drawing, interaction, and semantics decisions. Teach students to read from the outside inward: size, placement, visual treatment, then interaction and accessibility. Order matters; a clickable modifier placed on a different node can change the semantics and touch area.

```
Text(  
    text = "Continue",  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(horizontal = 16.dp)  
        .clip(RoundedCornerShape(12.dp))  
        .background(MaterialTheme.colorScheme.primary)  
        .clickable(onClick = onContinue)  
        .padding(vertical = 14.dp),  
    color = MaterialTheme.colorScheme.onPrimary,  
)
```

4.4 Layout and Material 3

Students should practise Row, Column, Box, LazyColumn, LazyVerticalGrid, Scaffold, TopAppBar, NavigationBar, Card, Button, OutlinedTextField, and Snackbar. Material 3 gives a coherent system, but it does not choose the product hierarchy for you. Ask students to name the primary action before they style it.

Studio task · Build the catalogue screen

Create a catalogue with a top app bar, a list of three courses, a loading placeholder, an empty state, and an error state. The list item must expose one primary action, preserve readable spacing at a narrow width, and provide a preview. Do not connect a database yet; use a local list so the UI can be evaluated separately from data access.

4.5 Unit 3 check

A student passes when they can explain recomposition, build a composable with a clear input/output contract, use modifiers deliberately, choose an appropriate layout, and show at least three non-success states without duplicating the screen structure.

5 Unit 4: State, events, forms, and ViewModel

5.1 State is the present truth

A screen becomes understandable when its state has a name, an owner, and a transition rule. Examples include the typed email, selected course, current request status, and last error message. Avoid scattering state across unrelated composables.

```
@Composable
fun TutorRequestForm(
    state: TutorRequestUiState,
    onEvent: (TutorRequestEvent) -> Unit,
) {
    OutlinedTextField(
        value = state.notes,
        onValueChange = { onEvent(TutorRequestEvent.NotesChanged(it)) },
        label = { Text("What support do you need?") },
        isError = state.notesError != null,
        supportingText = { state.notesError?.let { Text(it) } },
        modifier = Modifier.fillMaxWidth(),
    )
    Button(
        onClick = { onEvent(TutorRequestEvent.Submit) },
        enabled = state.canSubmit && !state.isSubmitting,
    ) {
        Text(if (state.isSubmitting) "Sending..." else "Request tutor")
    }
}
```

5.2 State hoisting

State hoisting moves state to the lowest common owner that needs to read or change it. A field can own temporary cursor behaviour, while the screen or ViewModel owns the submitted form. Hoist only what must be shared; do not create a global state object for every text field.

5.3 ViewModel and unidirectional data flow

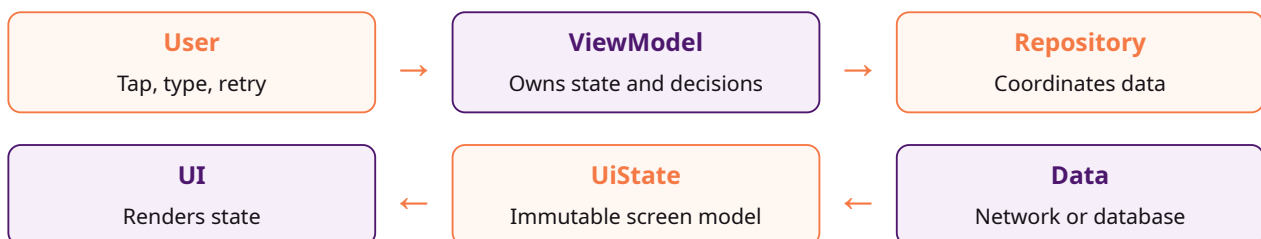


Figure 3: Events down, state up: a practical screen architecture.

```
data class CoursesUiState(
    val isLoading: Boolean = false,
    val courses: List<Course> = emptyList(),
    val errorMessage: String? = null,
)
```

```
class CoursesViewModel(  
    private val repository: CoursesRepository,  
) : ViewModel() {  
    private val _uiState = MutableStateFlow(CoursesUiState())  
    val uiState: StateFlow<CoursesUiState> = _uiState.asStateFlow()  
  
    fun refresh() {  
        viewModelScope.launch {  
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }  
            runCatching { repository.getCourses() }  
                .onSuccess { rows ->  
                    _uiState.value = CoursesUiState(courses = rows)  
                }  
                .onFailure { error ->  
                    _uiState.value = CoursesUiState(errorMessage = error.message)  
                }  
        }  
    }  
}
```

Explain why `viewModelScope` matters: the operation is tied to the `ViewModel` lifecycle rather than to a single recomposition. Students should not launch work directly from a composable body.

5.4 Lab 4 • Form state machine

Build a learner profile form with name, email, and learning goal. Required states are: initial; editing; invalid; submitting; saved; and server error. Students must draw the state machine, implement it, and write a short note explaining which state survives rotation and which input is intentionally temporary.

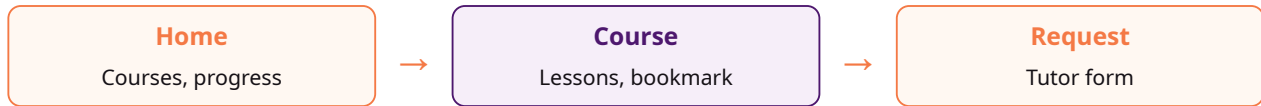
5.5 Unit 4 check

A student passes when they can identify the owner of state, send events upward, render state downward, prevent duplicate submission, and explain why a `ViewModel` is a better owner for screen-level work than a composable body.

6 Unit 5: Navigation and application architecture

6.1 Navigation is a product decision

Navigation answers where the user can go, what information must be carried forward, and what happens when they press Back. A navigation graph is not merely a collection of routes. It expresses the supported journeys of the product.



Back returns to the previous meaningful context; deep links must restore the right record.

Figure 4: A small LEA Connect navigation graph.

```

@Composable
fun LeaNHost(navController: NavController) {
    NavHost(
        navController = navController,
        startDestination = "home",
    ) {
        composable("home") {
            HomeScreen(onOpenCourse = { id ->
                navController.navigate("course/$id")
            })
        }
        composable(
            route = "course/{courseId}",
            arguments = listOf(navArgument("courseId") {
                type = NavType.StringType
            }),
        ) { entry ->
            CourseScreen(
                courseId = entry.arguments?.getString("courseId").orEmpty(),
                onRequestTutor = { id ->
                    navController.navigate("tutor/$id")
                },
            )
        }
        composable("tutor/{courseId}") { entry ->
            TutorRequestScreen(
                courseId = entry.arguments?.getString("courseId").orEmpty(),
            )
        }
    }
}

```

Never pass an entire mutable object through a route when a stable identifier is enough. Load the record from the appropriate state holder or repository, validate access, and show a clear not-found state.

6.2 Layers and responsibility

A practical Android architecture contains a UI layer, a data layer, and optionally a domain layer. The UI layer renders state. ViewModels translate user events into decisions. Repositories coordinate remote and local data sources. Domain use cases are useful when a rule is reused or deserves independent testing.

| Layer | Owns | Example | Must avoid | |—|—|—|—| | UI | Layout, semantics, visual states | CourseScreen | Raw database writes and secret keys | | ViewModel | Screen state and events | CoursesViewModel | Pixel-level layout decisions | | Domain | Reusable rules | QuoteTutorPrice | Android-specific rendering | | Repository | Data coordination | CoursesRepository | UI copy and navigation | | Data source | One source mechanism | CoursesApi, CourseDao | Deciding product policy |

Studio task · Architecture map

Draw your capstone's UI, ViewModel, domain, repository, API, and database boundaries. For each arrow, write what is allowed to cross it. Then implement one navigation route using an ID argument and a not-found state.

6.3 Unit 5 check

A student passes when they can defend a navigation graph, explain why route arguments should be stable identifiers, and place a new feature in the correct layer without hiding business rules inside a composable.

7 Unit 6: Coroutines, Flow, and network data

7.1 Asynchronous work is normal

A network call can take milliseconds, seconds, or never complete. Coroutines let students express asynchronous work without blocking the main thread. Flow represents a stream of values over time, which is useful for database updates, connectivity, and UI state.

```
class CoursesRepository(
    private val api: CoursesApi,
) {
    suspend fun getCourses(): List<Course> {
        return api.getCourses().items.map { it.toDomain() }
    }
}

class CoursesViewModel(
    private val repository: CoursesRepository,
) : ViewModel() {
    val courses: StateFlow<List<Course>> = flow {
        emit(repository.getCourses())
    }.catch { emit(emptyList()) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList(),
        )
}
```

Explain that suspend does not automatically mean “background thread.” The dispatcher and structured scope still matter. Teach students to keep network work away from the main thread and to make cancellation possible when the screen no longer needs it.

7.2 API contract and serialization

Before writing a client, document method, endpoint, request body, authentication, success shape, error shape, timeout, and retry policy. Use data-transfer objects at the boundary and map them to domain models so server naming does not leak through the application.

```
@Serializable
data class CourseDto(
    val id: String,
    val title: String,
    @SerializedName("price_kes") val priceKes: Int,
)

data class Course(
    val id: String,
    val title: String,
    val priceKes: Int,
)
```



```
fun CourseDto.toDomain() = Course(
    id = id,
    title = title,
    priceKes = priceKes,
)
```

7.3 The five-state screen

Every network-backed screen should have at least: loading, success with records, success with no records, recoverable error, and unauthorised or forbidden state. A retry action should not erase typed user input unless the user asks it to.

State	What the learner sees	Teacher observation prompt
Loading	Progress indicator and preserved context	Can the learner tell that work is happening?
Success	Useful data and next action	Does the hierarchy support the task?
Empty	Honest explanation and a useful first action	Does “nothing yet” still guide the user?
Error	Cause in plain language, retry, support path	Can a user recover without guessing?
Unauthorised	Sign-in or permission explanation	Does the boundary protect data without dead-ending the user?

7.4 Lab 6 · Remote catalogue

Students connect the catalogue screen to a mock or real endpoint. They must include a fake slow response for testing, a retry button, a safe empty state, and a redacted Logcat statement that does not print tokens or personal data. They submit the API contract and a screen recording or test evidence for each state.

7.5 Unit 6 check

A student passes when they can explain cancellation, distinguish a one-shot result from a Flow, map DTOs to domain models, and implement a remote screen that remains understandable when the network is slow or unavailable.

8 Unit 7: Room persistence and offline-first thinking

8.1 Why local storage matters

A serious mobile application must decide what it can show when the network is unavailable. Room provides a structured SQLite abstraction with entities, DAOs, queries, migrations, and observable results. Teach persistence as a product decision: what is cached, what is authoritative, how long data remains, and what happens when records conflict.

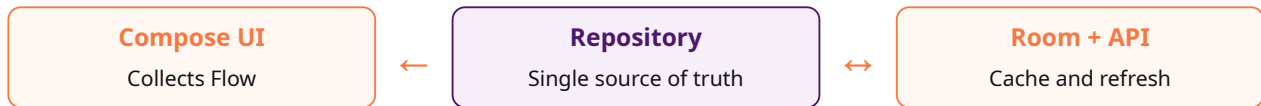


Figure 5: Repository coordination between local and remote sources.

```

@Entity(tableName = "lessons")
data class LessonEntity(
    @PrimaryKey val id: String,
    val courseId: String,
    val title: String,
    val completed: Boolean,
    val updatedAt: Long,
)

@Dao
interface LessonDao {
    @Query("SELECT * FROM lessons WHERE courseId = :courseId ORDER BY updatedAt DESC")
    fun observeLessons(courseId: String): Flow<List<LessonEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertAll(lessons: List<LessonEntity>)

    @Query("UPDATE lessons SET completed = :completed WHERE id = :lessonId")
    suspend fun setCompleted(lessonId: String, completed: Boolean)
}
  
```

Use an abstract RoomDatabase class and version it deliberately. A schema change is not a reason to delete the database during a real release. Students should write a migration note even if their first prototype only has version 1.

8.2 Repository pattern

```

class LessonRepository(
    private val dao: LessonDao,
    private val api: LessonsApi,
) {
    fun observeLessons(courseId: String): Flow<List<Lesson>> =
        dao.observeLessons(courseId).map { rows -> rows.map { it.toDomain() } }

    suspend fun refresh(courseId: String) {
        val remote = api.getLessons(courseId)
    }
}
  
```

```
        dao.upsertAll(remote.items.map { it.toEntity() })
    }
}
```

The UI observes the database. Refresh updates the database. This gives one source of truth for the screen and makes offline display easier to reason about.

8.3 Lab 7 • Offline lesson tracker

Build a lesson list that works with airplane mode after one successful sync. A student must demonstrate: initial empty state, synced state, marking a lesson complete, closing and reopening the app, a failed refresh that leaves cached data visible, and a migration note. The teacher should inspect whether the learner can explain why the UI reads Room rather than directly reading the network response.

8.4 Unit 7 check

A student passes when they can define an entity, write a DAO query, explain `Flow<List<T>>`, show a repository that coordinates data sources, and describe a safe migration or deletion policy.

9 Unit 8: Material 3, adaptive layouts, and design systems

9.1 Visual systems reduce accidental decisions

A design system gives the application consistent colour, typography, shape, spacing, interaction, and accessibility decisions. LEA Labs uses a dark purple foundation, brand purple, orange accents, lavender surfaces, and near-black text. The point is not to copy a brand palette mechanically; it is to make a system that supports hierarchy and readability.

```
private val LeaPurple = Color(0xFF4D176E)
private val LeaOrange = Color(0xFFFF47945)
private val LeaDark = Color(0xFF1F0D2E)
private val LeaLavender = Color(0xFFFF6EEF9)

private val LeaColorScheme = lightColorScheme(
    primary = LeaPurple,
    secondary = LeaOrange,
    background = Color(0xFFFFFDFB),
    surface = LeaLavender,
    onPrimary = Color.White,
    onSecondary = Color.White,
)

@Composable
fun LeaTheme(content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = LeaColorScheme,
        typography = Typography(),
        content = content,
    )
}
```

Teach contrast and hierarchy before decoration. Orange is useful for a primary action or feedback accent; it should not become a wall of attention competing with the page title.

9.2 Adaptive thinking

A phone, tablet, foldable, and desktop window can expose the same product through different layouts. Ask what the user needs at each width rather than stretching a phone layout until it breaks. A list-detail pattern may use one screen on a phone and two panes on a tablet.

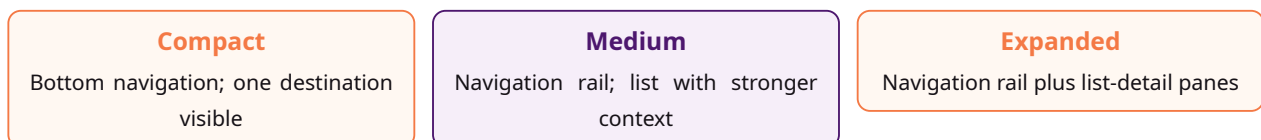


Figure 6: One content model, multiple window strategies.

```
@Composable
fun CourseLayout(
    windowSize: WindowWidthSizeClass,
    courseList: @Composable () -> Unit,
    detail: @Composable () -> Unit,
```

```
) {  
    if (windowSize == WindowWidthSizeClass.Expanded) {  
        Row(Modifier.fillMaxSize()) {  
            Box(Modifier.weight(0.38f)) { courseList() }  
            VerticalDivider()  
            Box(Modifier.weight(0.62f)) { detail() }  
        }  
    } else {  
        detail()  
    }  
}
```

9.3 Lab 8 · Adaptive redesign

Take the Unit 3 catalogue and produce compact and expanded variants. Use the emulator's resizable window or a tablet profile. Students must record one decision that changes by width and one design rule that remains constant. Add an accessibility check for larger text.

9.4 Unit 8 check

A student passes when they can identify design tokens, explain why a layout should adapt, use a Material 3 component with a deliberate hierarchy, and demonstrate that increased text size does not destroy the primary task.

10 Unit 9: Dependency injection and reusable data boundaries

10.1 Why injection matters

A ViewModel that constructs its own API client, database, clock, and analytics service is difficult to test and difficult to change. Dependency injection supplies collaborators from outside. The student does not need to memorise every Hilt annotation on day one; the essential idea is that a class declares what it needs and a composition root decides how to provide it.

```
class QuoteTutorPrice(
    private val pricing: TutorPricing,
) {
    fun quote(mode: SessionMode, packageSize: Int): Int {
        return pricing.amountFor(mode = mode, packageSize = packageSize)
    }
}

class FakeTutorPricing : TutorPricing {
    override fun amountFor(mode: SessionMode, packageSize: Int): Int =
        if (mode == SessionMode.ONLINE) 1800 else 2500
}
```

The fake implementation makes a pure unit test possible. In production, a real implementation can read a configuration or a backend policy without changing the rule's caller.

10.2 Hilt-shaped composition root

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    @Singleton
    fun provideCoursesApi(): CoursesApi = Retrofit.Builder()
        .baseUrl("https://example.test/")
        .addConverterFactory(Json.asConverterFactory("application/
json".toMediaType()))
        .build()
        .create(CoursesApi::class.java)
}
```

Do not place private production secrets in the app. A base URL may be public; an administrative token, signing key, or database credential may not be. Teach the difference between configuration and secret material.

10.3 Lab 9 • Replaceable collaborators

Students refactor one ViewModel so it receives a repository through its constructor. They add a fake repository that returns a known list and a failing repository that throws. The UI test must be able to render success and error without the network.

10.4 Unit 9 check

A student passes when they can explain constructor injection, replace a real dependency with a fake, and identify which values are safe to ship in an APK and which values must remain server-side.

11 Unit 10: Background work with WorkManager

11.1 Work that outlives a screen

Some tasks should continue after the user leaves a screen: uploading a queued assignment, refreshing a cache, exporting a report, or retrying a notification. WorkManager is appropriate for deferrable, persistent work that should survive process or device restarts. It is not a substitute for every coroutine and it is not the right choice for a visible, immediate interaction.

```
class LessonSyncWorker(
    appContext: Context,
    params: WorkerParameters,
    private val repository: LessonRepository,
) : CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        return runCatching {
            repository.refresh(courseId =
                inputData.getString("courseId").orEmpty())
        }.fold(
            onSuccess = { Result.success() },
            onFailure = { Result.retry() },
        )
    }
}
```

In a real project, use a supported dependency-injection integration or a worker factory rather than pretending that a constructor dependency appears automatically. The teaching point is lifecycle ownership, retry policy, and observability.

```
val request = OneTimeWorkRequestBuilder<LessonSyncWorker>()
    .setInputData(workDataOf("courseId" to courseId))
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build()
    )
    .build()

WorkManager.getInstance(context).enqueueUniqueWork(
    "lesson-sync-$courseId",
    ExistingWorkPolicy.KEEP,
    request,
)
```

11.2 Lab 10 · Reliable refresh

Students schedule a unique refresh, test it under no network, inspect the result, and explain why duplicate work should not be queued for the same course. They write a short retry and backoff policy note.

11.3 Unit 10 check

A student passes when they can distinguish immediate UI work from persistent background work, choose an appropriate constraint, explain idempotence, and show observable success or failure.

12 Unit 11: Testing, debugging, and performance

12.1 A layered quality strategy

Quality is built through several forms of evidence. Static checks catch malformed code and type errors. Unit tests verify pure rules. Compose UI tests verify semantics and interaction. Integration tests verify multiple boundaries. Scenario testing asks whether a person can accomplish a real task on a real device.

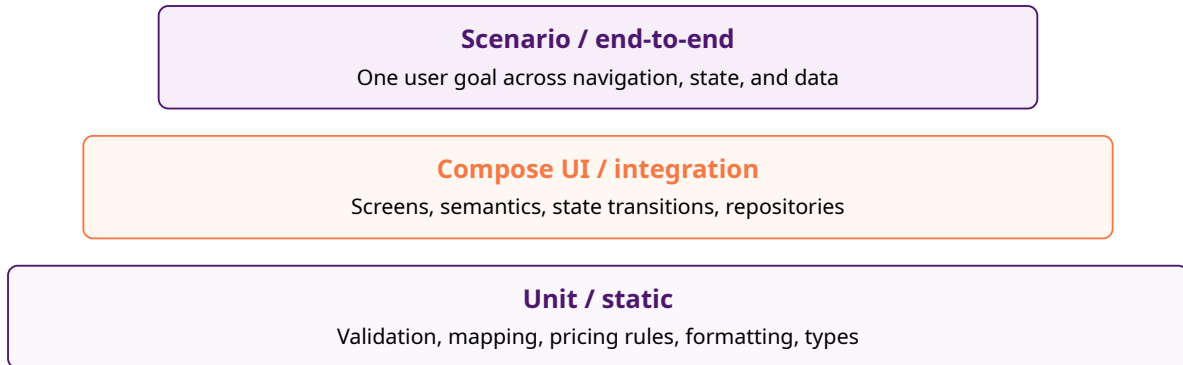


Figure 7: Testing layers for a small Android application.

12.2 Unit test example

```
class TutorPriceTest {
    @Test
    fun faceToFaceSingleSessionUsesThePublishedPrice() {
        val pricing = FakeTutorPricing()
        val useCase = QuoteTutorPrice(pricing)

        val amount = useCase.quote(SessionMode.FACE_TO_FACE, packageSize = 1)

        assertEquals(2500, amount)
    }
}
```

A good test name states behaviour, not implementation. The arrange-act-assert structure makes the reasoning visible.

12.3 Compose UI test example

```
@get:Rule
val composeRule = createComposeRule()

@Test
fun submitButtonIsDisabledUntilRequiredNoteExists() {
    composeRule.setContent {
        TutorRequestForm(
            state = TutorRequestUiState(notes = "", canSubmit = false),
            onEvent = {},
        )
    }

    composeRule
```



```
        .onNodeWithText("Request tutor")
        .assertIsNotEnabled()
    }
```

Semantics are the bridge between UI meaning and automated interaction. Avoid tests that depend only on fragile screen coordinates.

12.4 Debugging protocol

When a test or user report fails, record: expected result; observed result; smallest reproduction; device and build; logs without secrets; first likely boundary; and the next experiment. Change one relevant variable at a time. A good teacher models calm investigation rather than random deletion of code.

12.5 Performance conversation

Teach students to look for unnecessary recomposition, large work on the main thread, unbounded lists, images without sizing, repeated network requests, and memory retained longer than needed. Use Android Studio's profiler and Layout Inspector to ask questions, not merely to collect screenshots.

12.6 Lab 11 · Evidence-led repair

Give each student a seeded defect: missing loading state, unlabelled button, duplicate request, slow computation on the main thread, or stale cached record. Require a bug report, a hypothesis, one repair, one regression test, and a short explanation of why the repair belongs at that boundary.

12.7 Unit 11 check

A student passes when they can select a suitable test layer, write a behaviour-focused test, use semantics rather than coordinates, and document a repair as a reproducible engineering change.

13 Unit 12: Accessibility, privacy, and secure application behaviour

13.1 Accessibility is a design requirement

An app is not finished when it looks correct for the developer. Students must consider TalkBack, text scaling, contrast, touch targets, focus order, descriptive labels, error recovery, motion sensitivity, and different physical contexts. Compose semantics make meaning inspectable, but the developer must still supply meaningful meaning.

```
IconButton(
  onClick = onBookmark,
  modifier = Modifier.semantics {
    contentDescription = if (isBookmarked) {
      "Remove bookmark from ${lesson.title}"
    } else {
      "Bookmark ${lesson.title}"
    }
    stateDescription = if (isBookmarked) "Bookmarked" else "Not bookmarked"
  },
) {
  Icon(
    imageVector = if (isBookmarked) Icons.Filled.Bookmark else
Icons.Outlined.BookmarkBorder,
    contentDescription = null,
  )
}
```

The decorative icon is hidden because the button already exposes the complete action. The state description tells assistive technology what changed.

13.2 Privacy and authorisation

Authentication identifies the user. Authorisation checks what that user may do. Never rely on a hidden button as the only permission check. The server or trusted data boundary must enforce ownership and role rules. Store the minimum personal data required, explain retention, and avoid logging tokens or full private messages.

| Risk | Weak approach | Better teaching standard | |—|—|—| | Secret leakage | Hard-code an API key in the app | Keep secrets server-side; use scoped public configuration only. | | Cross-account access | Trust an ID sent by the client | Derive ownership from the authenticated session and verify it server-side. | | Sensitive logs | Print tokens and full payloads | Redact identifiers, tokens, and private content. | | Insecure local data | Store credentials in plain text | Use platform-secure storage and avoid storing what is not needed. | | Unsafe links | Open arbitrary URLs without validation | Validate destinations and explain external hand-off. |

13.3 Lab 12 · Accessibility and privacy audit

Students audit their capstone using a checklist: every interactive element has a meaningful label; content is usable at 200% text size; colour is not the only signal; keyboard or switch navigation has a sensible order; errors are recoverable; personal data is minimised; permissions are requested in context; and logs are safe. They must fix three findings and document one limitation honestly.

13.4 Unit 12 check

A student passes when they can inspect semantics, explain how a screen changes for assistive technology, distinguish authentication from authorisation, and identify at least two ways their app could expose or retain too much information.

14 Unit 13: Capstone planning and product discovery

14.1 Build one useful layer at a time

A capstone is not a contest to include the most screens. It is an opportunity to prove that a team can move from a defined user and task to a working, explainable release. The first planning artefact is a brief.

14.2 One user, one job, one proof

Write the user in context, the job as a verb, and the proof as observable evidence. For example: "A learner between classes wants to request a face-to-face tutor session; proof is a saved request with a quoted price and an administrator response visible in the learner calendar."

User: _____
 Context: _____
 Job: I want to _____
 Non-goals: _____
 Success: We will know it works when _____
 Risk: _____

14.3 Product requirements and acceptance criteria

Turn wishes into behaviour. "The app should be easy" is not testable. "A new learner can find a course, open its details, and see a clear next action in under two minutes without assistance" can be tested.

| Feature | User story | Acceptance criteria | |—|—|—| | Course catalogue | As a learner, I want to browse courses so that I can choose a direction. | List loads; empty state exists; tap opens details; errors can retry. | | Tutor request | As an enrolled learner, I want to request support so that I can keep moving. | Mode, package, date, time, and notes validate; quote is visible; save is confirmed. | | Admin response | As an administrator, I want to review requests so that I can confirm or decline responsibly. | Request list is protected; response stores status and message; confirmed class appears to learner. |

14.4 Story mapping



Figure 8: A capstone story map from discovery to release.

14.5 Lab 13 · Proposal clinic

Teams submit a one-page brief, a five-to-eight-step user flow, a screen inventory, a data contract, an architecture map, and a two-week build plan. The teacher holds a short review focused on scope. Reject extra features that do not support the one primary job.

14.6 Unit 13 check

A student passes when they can explain the user and job without jargon, state non-goals, write acceptance criteria, and identify the smallest slice that can be built and tested first.

15 Unit 14: Capstone implementation, review, and release

15.1 Implementation sequence

Use vertical slices. A vertical slice moves one useful path through UI, state, data, and test rather than building all screens first. For LEA Connect, the first slice can be “view a course and bookmark one lesson.” The second can be “submit a tutor request.” The third can be “show an administrator response.”

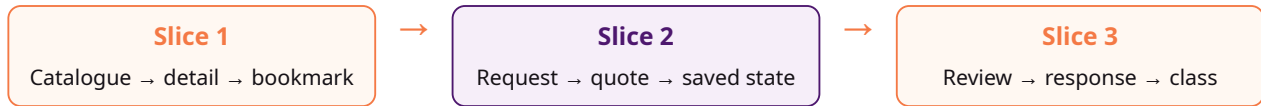


Figure 9: Vertical slicing keeps every iteration demonstrable.

15.2 Code review protocol

Every team should be able to answer: Where is state owned? Where is validation enforced? What happens if the network fails? Which records are persisted? How is access controlled? What test proves this? What would you change if the product had ten times as many users?

Use a review conversation rather than a score-only exchange. Start with the intended behaviour, ask the student to show the evidence, then discuss one strength, one risk, and one next experiment.

15.3 Demonstration structure

A professional five-minute demonstration follows this sequence: introduce the user and task; show the primary journey; show one empty or error state; show persisted or synchronised data; explain one architecture decision; demonstrate one test or accessibility check; state one known limitation and the next step.

15.4 Release checklist

Area	Question	Status
Product	Can the defined user understand the value and complete the primary job?	☐
Build	Can another student clone, sync, run, and inspect the project?	☐
Quality	Do unit, Compose, scenario, and device checks provide evidence?	☐
Accessibility	Are semantics, scale, contrast, focus, and recovery addressed?	☐
Security	Are permissions and secrets protected at the trusted boundary?	☐
Data	Are storage, deletion, migration, and offline choices documented?	☐
Release	Is the version, changelog, support path, and rollback idea recorded?	☐
Learning	What evidence will decide the next iteration?	☐

15.5 Final assessment rubric

| Criterion | Developing | Capable | Strong | |—|—|—|—| | Problem framing | Broad idea, unclear user or scope. | Defined user, job, non-goals, and acceptance criteria. | Evidence changes the scope and the team can defend the trade-off. | | Kotlin and Compose | Code works with heavy copying or unclear ownership. | Functions, state, composables, and modifiers are understandable. | The code communicates boundaries and is easy to test or extend. | | Architecture | UI directly owns data and business decisions. | UI, ViewModel, repository, and data source responsibilities are visible. | Boundaries support substitution,

offline behaviour, and future growth. | | Interaction quality | Happy path only or weak feedback. | Loading, empty, error, validation, and success states are clear. | Recovery is calm, inclusive, and supported by evidence from observation. | | Data and security | Data rules are assumed or secrets are exposed. | Ownership and permissions are documented and enforced. | Privacy, retention, failure, and audit implications are explicitly considered. | | Testing | Few checks or screenshots without reasoning. | Tests cover rules, key UI behaviour, and real scenarios. | Tests guide design decisions and regressions are prevented. | | Communication | Demo describes screens. | Demo explains user, evidence, decisions, and limitations. | Team teaches another person how to run, inspect, and improve the product. |

15.6 Capstone submission package

Students submit the source project, README, product brief, flow and architecture map, data contract, screenshots or screen recording, test output, accessibility checklist, privacy note, release checklist, and a reflection of at least 500 words. The reflection must explain one decision that changed because of evidence.

16 Teacher field guide

16.1 A repeatable lesson pattern

A strong 120-minute studio can use this rhythm:

| Minutes | Teacher move | Student move | |—|—|—| | 0–10 | Connect today’s concept to the running project. | State the previous checkpoint and today’s goal. | | 10–25 | Explain one mental model with a diagram. | Predict what the code should do. | | 25–50 | Live-code a small slice, including one deliberate error. | Type, run, inspect, and ask why. | | 50–60 | Pause for a concept check. | Explain the boundary to a partner. | | 60–105 | Coach the lab through questions. | Build, test, record evidence. | | 105–120 | Review two artefacts and name next steps. | Submit a checkpoint and reflection. |

16.2 Questions that teach engineering

- What is the user trying to accomplish at this exact moment?
- Which state describes the screen now?
- Who owns that state and why?
- What event can change it?
- What is the smallest test that would catch this regression?
- What happens if the device rotates, the process dies, or the network disappears?
- What data is sensitive, and where is the trusted permission check?
- What would a learner using TalkBack hear here?
- What evidence would cause you to change the design?

16.3 Common misconceptions

| Misconception | Repair strategy | |—|—| | “Compose redraws the whole app every time.” | Use a small state change and inspect which composables recompose; explain the mental model without promising magic. | | “A ViewModel is a global database.” | Show that it owns screen-related state and delegates persistence to a repository. | | “A successful HTTP response means the feature works.” | Require loading, empty, error, permissions, persistence, and user evidence. | | “Accessibility is adding content descriptions at the end.” | Start with semantics and task flow; test with larger text and assistive navigation. | | “Copying code is learning.” | Require prediction, modification, explanation, and a changed test. | | “More features means a better capstone.” | Score scope discipline and evidence, not surface area. |

16.4 Feedback language

Use feedback that points to an observable decision: “The error state tells me that loading failed, but it does not tell the user how to recover.” “This repository hides the data source well; now add a test that proves it can be replaced.” “The button is visible but its semantics do not explain whether it is selected.” This is more teachable than “make it cleaner.”

17 Student worksheets and lab checklists

17.1 Architecture worksheet

Product name: _____

Primary user: _____

Primary job: _____

UI screens:

1. _____
2. _____
3. _____

State holder: _____

Repository: _____

Remote source: _____

Local source: _____

Permission boundary: _____

One likely failure: _____

Test that proves recovery: _____

17.2 Screen-state worksheet

| Screen | Loading | Success | Empty | Error | Unauthorised | Recovery action | |—|—|—|—|—|—|—|

| Home | ☐ | ☐ | ☐ | ☐ | ☐ | _____ | | Detail | ☐ | ☐ | ☐ | ☐ | ☐ | _____ | |

Form | ☐ | ☐ | ☐ | ☐ | ☐ | _____ |

17.3 Pull-request checklist

- [] The change has a clear user outcome.
- [] The source of truth is named.
- [] State and events are understandable.
- [] Loading, empty, error, and success states are handled.
- [] Tests are added or updated.
- [] Accessibility semantics are checked.
- [] Personal data and secrets are not logged.
- [] The README or decision record is updated.
- [] The reviewer can run the change without private setup.

17.4 Studio reflection prompts

1. What did you believe before you ran the code?
2. What did the program actually do?
3. Which boundary became clearer after debugging?
4. What evidence supports your current design?
5. What would you test next with a real learner?

18 Glossary

| Term | Working definition | |—|—| | Activity | An Android component that provides a user-facing entry point; modern Compose apps often use one activity as a host for destinations. | | Composable | A Kotlin function that describes part of a UI and can participate in composition. | | Coroutine | A cancellable unit

of asynchronous work that can suspend without blocking a thread. | | DAO | Data-access object that defines database operations for Room. | | DTO | Data-transfer object used at a network or external-data boundary. | | Flow | A Kotlin stream that can emit values over time. | | Repository | A boundary that exposes data and coordinates one or more data sources. | | State hoisting | Moving state to a shared owner while passing values and events through a component contract. | | ViewModel | A lifecycle-aware state holder for screen-level data and events. | | WorkManager | A Jetpack API for deferrable persistent background work. | | UDF | Unidirectional data flow: events move toward state owners and state moves toward the UI. | | AAB | Android App Bundle, a publishing artefact from which device-specific APKs can be generated. |

19 References and further study

The curriculum is aligned with the following official materials. Links are included so teachers can refresh examples as Android Studio, Kotlin, and Jetpack libraries evolve.

- [1] [Android Basics with Compose course](#), Android Developers. Used for the progression from Kotlin and Android Studio setup through Compose UI, state, lists, Material Design, accessibility, tests, and projects.
- [2] [Install Android Studio](#), Android Developers. Used for environment requirements, SDK setup, emulator guidance, physical-device alternatives, and first-run configuration.
- [3] [Guide to app architecture](#), Android Developers. Used for UI/data/domain layering, repositories, ViewModels, unidirectional data flow, coroutines and flows, dependency injection, and adaptive architecture.
- [4] [Get started with Jetpack Compose](#), Android Developers. Used for declarative UI, state, recomposition, modifiers, semantics, adaptive UI, Material 3, navigation, accessibility, and testing.
- [5] [Accessibility in Jetpack Compose](#), Android Developers. Used for semantics, traversal order, scalable content, semantic merging, inspection, debugging, and accessibility testing.
- [6] [Kotlin coroutines on Android](#), Android Developers. Used for structured asynchronous work, lifecycle-aware scopes, and non-blocking application behaviour.
- [7] [Kotlin Flows on Android](#), Android Developers. Used for streams of state and observable data.
- [8] [Save data in a local database using Room](#), Android Developers. Used for entities, DAOs, Room database design, and local persistence.
- [9] [Navigation with Compose](#), Android Developers. Used for navigation graphs, destinations, arguments, and ViewModel integration.
- [10] [Task scheduling with WorkManager](#), Android Developers. Used for persistent, deferrable background work and constraints.
- [11] [Dependency injection with Hilt](#), Android Developers. Used for constructor injection, application components, and testable collaborators.

20 Closing note

The strongest Android developer is not the person who remembers the most API names. It is the person who can make a user's task clearer, choose a boundary that keeps the code explainable, test the behaviour that matters, protect the data entrusted to the product, and keep learning from evidence. Build the next useful layer, teach another person what you learned, and leave the project easier to understand than you found it.